Tidy practice

MA2003B Application of Multivariate Methods in Data Science

Team Members:

Raul Gomez
Adriana Reyna

August 4, 2025

MA2003B Tidy Practice

Tecnológico de Monterrey

The WHO dataset

Before starting, we need to load the required libraries and load the WHO dataset:

```
library("tidyverse")
library("here")
library("cowplot")
library("patchwork")
library("krulRutils")
library("ISLR2")
library("magrittr")

options(scipen = 999) # Disable scientific notation
data(who)
```

We now proceed to clean the WHO dataset. The main problem with this dataset is that it contains variables (like "new_sp_m014") that are not very clear and that contain more than one piece of information. So we need to separate these variables and introduce better names for them and their associated values.

```
# We start by creating the variable "who_tidy"
#that contains the cleaned WHO dataset
who_tidy <- who %>%
  # We use pivot_longer to eliminate the variables starting with "new"
  # and use them as values instead
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) %>%
  separate(key, into = c("new", "type", "sexage"), sep = "_") %>%
  separate(sexage, into = c("sex", "age"), sep = 1) %>%
  select(-new, -iso2, - iso3)  %T>%
  # Finally, we save the tidy data in both RDS and CSV formats
```

```
  saveRDS(here("data", "who_tidy.rds")) %>%
  write_csv(here("data", "who_tidy.csv")) %>%
  glimpse()
```

```
Rows: 76,046
Columns: 6
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

We now want to convert the columns "type", "sex", and "age" into factors. We start by constructing the following lookup tables:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "Extrapulmonary TB",
    "Relapse case",
    "Smear-Negative pulmonary TB",
    "Smear-Positive pulmonary TB"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Female", "Male")
)

who_age_lookup_tbl <- tibble(
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
```

```
    "15-24",
    "25-34",
    "35-44",
    "45-54",
    "55-64",
    "65+"
  )
)
```

We can now append factor columns for the corresponding variables in the WHO dataset:

```
who_factor <- who_tidy %>%
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  glimpse()
```

```
Rows: 76,046
Columns: 9
$ country     <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan"~
$ year        <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997~
$ type        <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp"~
$ sex         <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f"~
$ age         <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014"~
$ cases       <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128~
$ type_factor <fct> Smear-Positive pulmonary TB, Smear-Positive pulmonary TB, ~
$ sex_factor  <fct> Male, Male, Male, Male, Male, Male, Male, Female, Female, ~
$ age_factor  <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24,~
```

We now generate the associated plot for the cleaned WHO dataset. We want to generate a bar plot showing the number of cases grouped by type and sex. To start with, I generate a tibble that contains this information:

```
who_type_sex_tbl <- who_factor %>%
  count(type_factor, sex_factor, wt = cases, name = "cases")
```

We can now use this tibble to generate the required plot:

```
who_type_sex_plot <- who_type_sex_tbl %>%
  ggplot(aes(x = type_factor, y = cases, fill = sex_factor)) +
  geom_col(position = "dodge") +
  scale_fill_manual(
    values = c(
      "Female" = "#e7298a",
      "Male" = "#1f77b4"
    )
  ) +
  labs(
    title = "Tuberculosis Cases by Type and Sex",
    x = "Tuberculosis Type",
    y = "Cases",
    fill = "Sex"
  ) +
  theme_krul()
```

We can now save and plot the graph:

```
ggsave(
  filename = here("images", "who_plot.png"),
  plot = who_type_sex_plot,
  width = 12,
  height = 6,
  dpi = 300
)
who_type_sex_plot
```

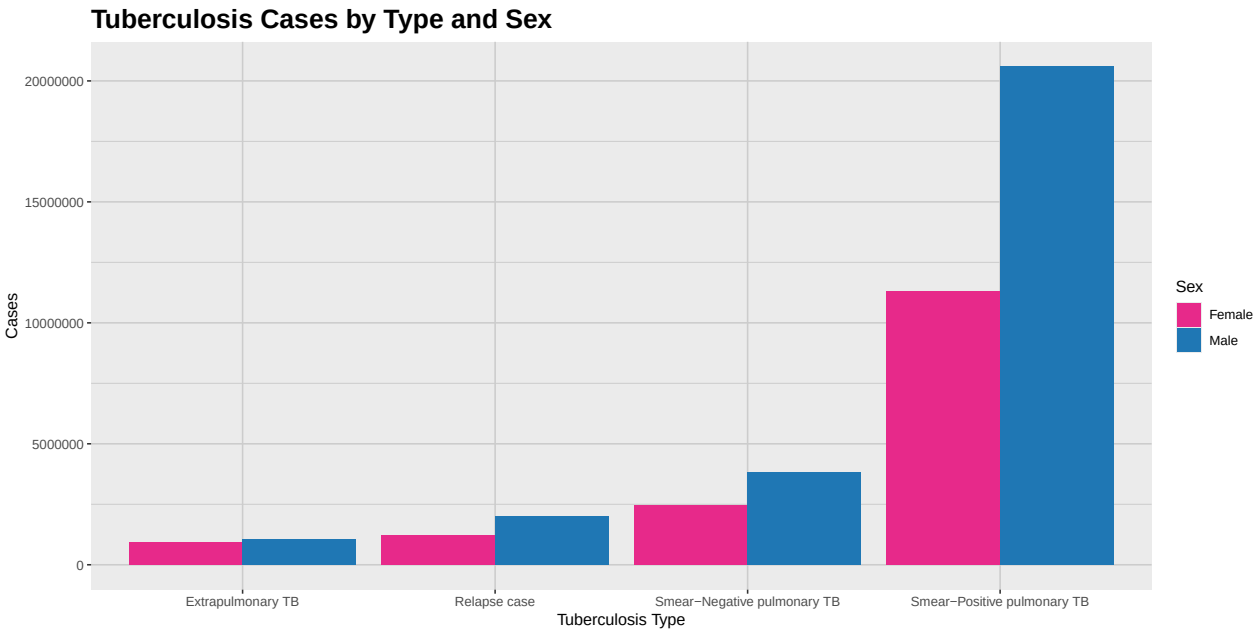

Figure 1: This plot shows a comparative analysis of the number of cases of tuberculosis by type and sex. As we can see from the plot, the number of cases is consistently higher for the male population across all types of tuberculosis.

The Billboard dataset

We now proceed to clean the Billboard dataset. The problem with this dataset is that the weeks in which a song appeared in the Billboard chart are represented as columns, which makes it difficult to analyze the data. So we want to move this information to a single column called "week".

```
# We first make sure to load the Billboard dataset
data(billboard)
# Now we create the variable "billboard_tidy"
billboard_tidy <- billboard %>%
  # We use pivot_longer to move the weeks into a single column
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "ranking",
    values_drop_na = TRUE
  ) %>%
  # We now clean the "week" variable to remove the "wk" prefix
  # leaving only the week number
  mutate(
    week = if_else(
```

```
      startsWith(week, "wk"),
      sub("wk", "", week),
      week
    )
  ) %>%
  # We make sure that the "week" variable is an integer
  # We also add the "date" variable
  # which gives the date corresponding to each week
  # in which the song appeared in the Billboard chart
  mutate(
    week = as.integer(week),
    date = date.entered + weeks(week - 1)
  ) %>%
  # Now we arrange the data by date and ranking for easy reference
  arrange(date, ranking) %T>%
  # Finally, we save the tidy data in both RDS and CSV formats
  saveRDS(here("data", "billboard_tidy.rds")) %>%
  write_csv(here("data", "billboard_tidy.csv"))
```

We now want to generate a plot that keeps track of the ranking of the song "who let the dogs out", by "Baha Men", in the Billboard chart. We start by filtering the dataset to keep only the relevant song:

```
dogs_out_tbl <- billboard_tidy %>%
  filter(track == "Who Let The Dogs Out") %>%
  arrange(week) %>%
  select(date, ranking)
```

With this tibble, we can now generate the required plot:

```
dogs_out_plot <- dogs_out_tbl %>%
  ggplot(aes(x = date, y = ranking)) +
  geom_line(color = "#1f77b4", linewidth = 1) +
  geom_point(color = "#d62728", size = 2) +
  scale_x_date(date_labels = "%B %Y") +
  scale_y_reverse(breaks = seq(0, 100, 10)) + # rank 1 is at the top
  labs(
    title = "Chart Performance of 'Who Let The Dogs Out' by 'Baha Men'",
    x = "Date",
    y = "Billboard Hot 100 Ranking"
  ) +
  theme_krul()
```

Finally, we can save and plot the graph:

```
ggsave(
  filename = here("images", "dogs_out_plot.png"),
  plot = dogs_out_plot,
  width = 12,
  height = 6,
  dpi = 300
)

dogs_out_plot
```

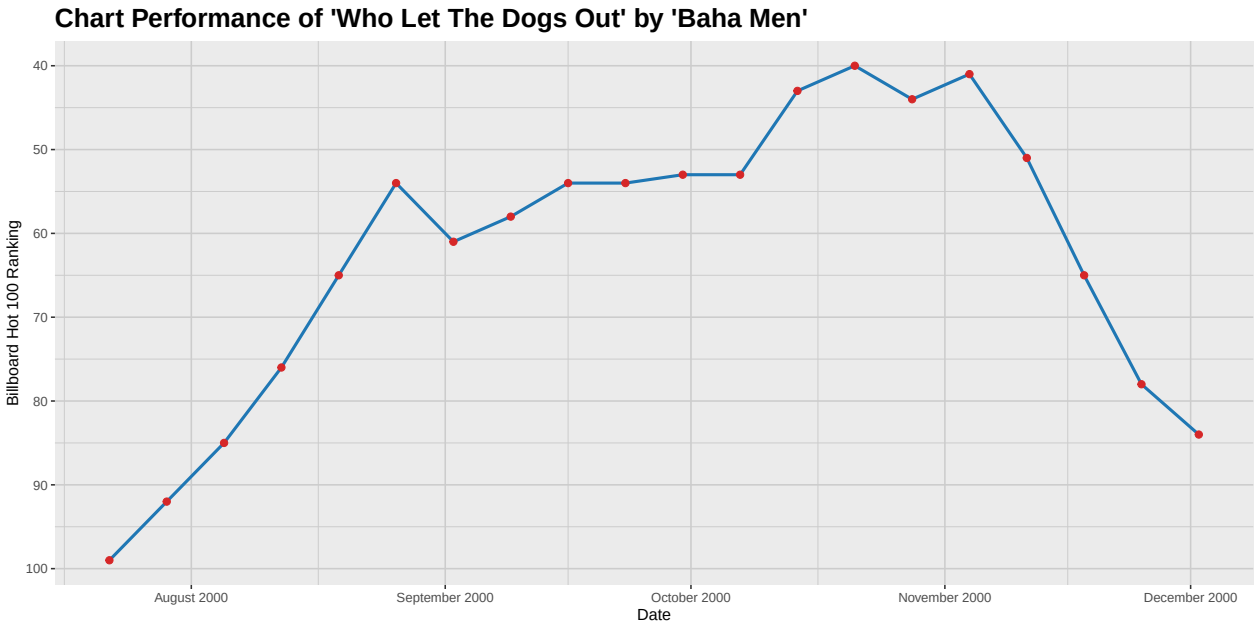

Figure 2: In this plot we can see the evolution of the ranking of the song 'Who Let The Dogs Out' by 'Baha Men' in the Billboard Hot 100 chart. The song peaked at rank 40 in the week of October 21, 2000, and remained in the top 100 for several weeks.

The Pew dataset

Finally, we will clean the Pew dataset. Since I was not able to find the original dataset without a subscription, I will use this artifially created dataset that contains the same structure as the original one.

```
pew <- tibble(
  religion = c(
```

```
    "Agnostic", "Atheist", "Buddhist", "Catholic", "Don't know/refused",
    "Evangelical Prot", "Hindu", "Historically Black Prot", "Jehovah's Witness",
    "Jewish", "Mainline Prot", "Mormon", "Muslim", "Orthodox",
    "Other Christian", "Other Faiths", "Other World Religions", "Unaffiliated"
  ),
  `$10k` = c(
    27, 12, 27, 418, 15, 575, 1, 228, 20, 19, 289, 29, 6, 13, 9, 20, 5, 217
  ),
  `$10-20k` = c(
    34, 27, 21, 617, 19, 869, 9, 244, 27, 19, 495, 40, 7, 17, 11, 33, 2, 299
  ),
  `$20-30k` = c(
    60, 37, 30, 732, 28, 1064, 7, 236, 24, 25, 655, 48, 9, 17, 10, 40, 3, 374
  ),
  `$30-40k` = c(
    81, 52, 34, 670, 24, 982, 9, 238, 24, 31, 651, 51, 8, 14, 10, 51, 1, 365
  )
)
```

Now, the problem with this dataset is that the income categories are represented as columns, which makes it difficult to analyze the data. So we want to move this information to a single column called "income".

```
pew_tidy <- pew %>%
  pivot_longer(
    cols = -religion,
    names_to = "income",
    values_to = "count",
    values_drop_na = TRUE
  ) %>%
  uncount(weights = count) %T>%
  saveRDS(here("data", "pew_tidy.rds")) %>%
  write_csv(here("data", "pew_tidy.csv"))
```

Before generating the plot, we will change the names of the variables to make them shorter and generate the corresponding colors and factors:

```
income_colors <- c(
  "$30-40k" = "#1f77b4",
  "$20-30k" = "#2ca02c",
  "$10-20k" = "#ff7f0e",
  "$10k" = "#d62728"
```

```
)

income_levels <- c(
  "$30-40k",
  "$20-30k",
  "$10-20k",
  "$10k"
)

pew_tidy <- pew_tidy %>%
  mutate(
    religion = if_else(
      startsWith(religion, "Historically Black Prot"),
      sub("Historically Black Prot", "Black Prot", religion),
      religion
    )
  )


pew_tidy <- pew_tidy %>%
  mutate(
    religion = if_else(
      startsWith(religion, "Other World Religions"),
      sub("Other World Religions", "Other Religions", religion),
      religion
    )
  )

pew_tidy <- pew_tidy %>%
  mutate(income = factor(income, levels = income_levels, ordered = TRUE))
```

We can now generate the required table which groups the data by religion and income:

```
pew_grouped <- pew_tidy %>%
  count(religion, income, name = "count")
```

With this tibble, we can now generate the required plot:

```
pew_plot <- pew_grouped %>%
  ggplot(aes(
    x = fct_reorder(religion, count, .desc = TRUE),
    y = count,
    fill = income
```

```
  )) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = income_colors) +
  labs(
    title = "Distribution of income brackets per religion",
    x = "Religion",
    y = "Followers",
    fill = "Income Bracket"
  ) +
  theme_krul() +
  coord_flip()
```

Finally, we can save and plot the graph:

```
ggsave(
  filename = here("images", "pew_plot.png"),
  plot = pew_plot,
  width = 12,
  height = 12,
  dpi = 300
)

pew_plot
```

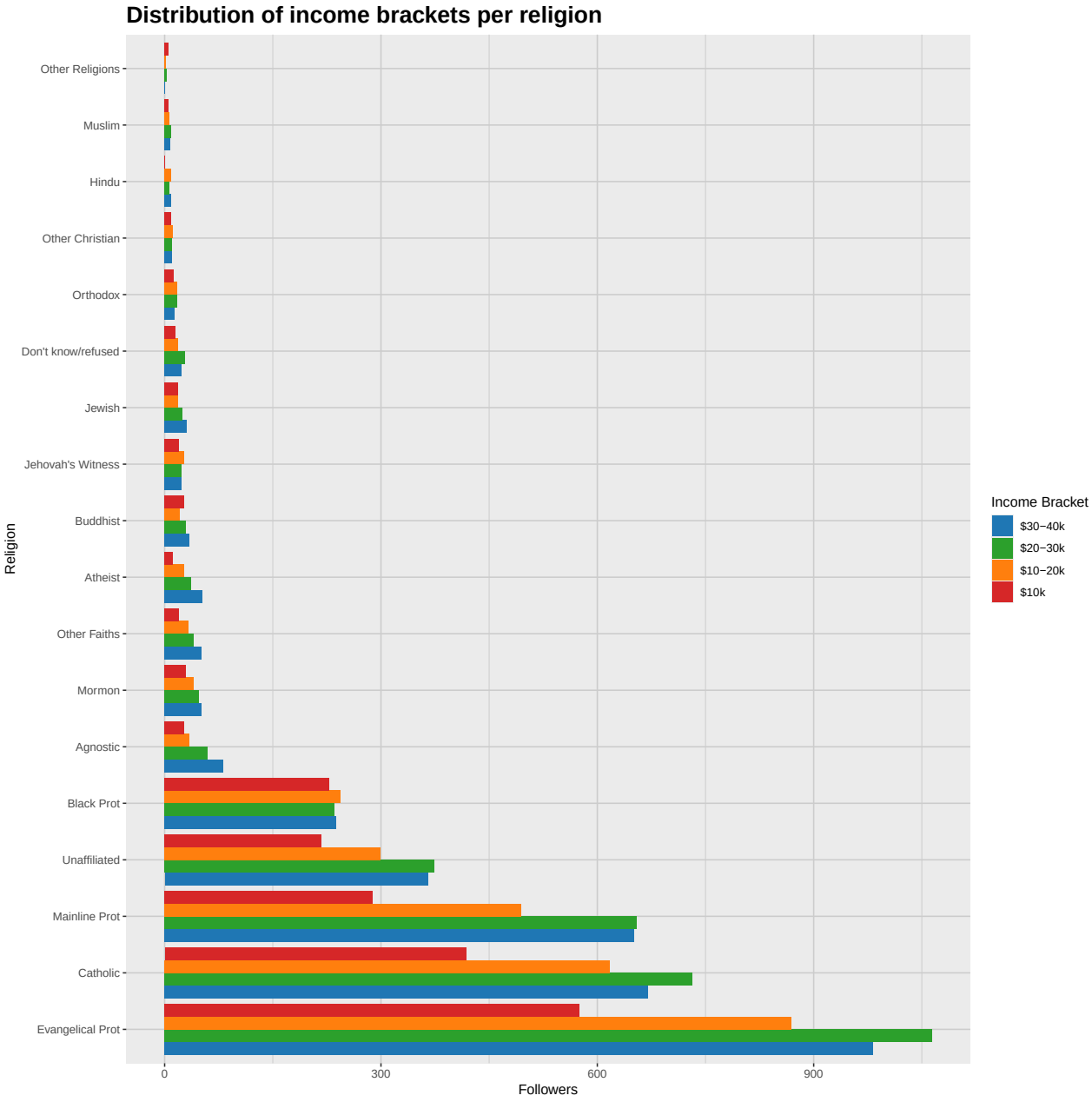

Figure 3: This plot shows the distribution of income brackets per religions. We can observe some differences in the distribution of these income brackets associated to the different social environments were the different religions thrive.

Conclusions

In this document, we have practiced cleaning datasets and generating basic plots using the "tidyverse" package in R. This covers the basics of "Exploratory Data Analysis", for discrete datasets. I still need to look into histograms and boxplots for continuous datasets. We have worked with three different datasets:

1. For the WHO dataset we generated a column plot and made sure that the bars for sex were separated.
2. For the Billboard dataset, we generated a plot that kept track of the performance of a given song.
3. For the Pew dataset, we generated a column plot, but in this case we stacked the bars for easy reference. (Although having them separated acutally looks good, and now that I have flipped the axese, it fits.) I also flipped the axes to make the plot more readable.